

Reviewer Pack — VC-Dimension of Row-Induced Set System Bounds Monotone DNF S...

Entry 91f293427310 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 10:09:36 UTC

VC-Dimension of Row-Induced Set System Bounds Monotone DNF Size for k-CLIQUE

Entry ID: 91f293427310 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 10:09:36 UTC

1. Conjecture

Field A (mathematical branch): Statistical learning theory (VC-theory) **Field B** (complexity object): Monotone DNF representation size

Statement:

Let F be a monotone DNF formula representing the k -CLIQUE function on n -vertex graphs. Let S_F be the set system induced by the truth-table rows of F , where each row corresponds to a graph and each term in F defines a set of graphs containing that term's edges. Then the VC-dimension of S_F is at least $\Omega(n)$ and any such DNF with VC-dimension v satisfies $|F| \geq 2^{\lfloor v/4 \rfloor}$.

Rationale (proposer's reasoning):

VC-dimension captures the expressive capacity of set systems and is tightly linked to sample complexity in learning theory. Monotone DNFs with few terms can only shatter small sets of variables, limiting their VC-dimension. The k -CLIQUE function requires distinguishing all k -subsets, which induces a large shattering, potentially exposing an information-theoretic bottleneck in compact representation.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 048486d81c2317d8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

VC-dimension bounds DNF size for k -CLIQUE

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -VC dimension AND monotone DNF AND clique function - statistical learning theory AND k clique AND DNF representation size - VC theory AND row induced set systems AND monotone Boolean formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=19.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_truth_table(n, k):
        truth_table = []
        for i in range(2 ** n):
            row = [bool((i >> j) & 1) for j in range(n)]
            if any(row[j] and not row[j + k] for j in range(n - k)):
                truth_table.append(row)
        return truth_table

    def is_shattered(S, S_F):
        for subset in powerset(S):
            if all(any(all(subset[i] == s[i] for i in range(len(subset))) for s in S_F) for subset
                return False
        return True

    def powerset(s):
        result = []
        for i in range(1 << len(s)):
            subset = [s[j] for j in range(len(s)) if (i & (1 << j))]
            result.append(subset)
        return result

    n = random.randint(10, 20)
    k = 3
    truth_table = generate_truth_table(n, k)

    S_F = [tuple(row) for row in truth_table]
    VC_dimension = 0
```

```

while True:
    VC_dimension += 1
    if not is_shattered(range(VC_dimension), S_F):
        break

DNF_size = len(truth_table)

return {
    "metric_name": "VC-dimension",
    "metric_value": VC_dimension,
    "instances_tested": 1,
    "conjecture_holds": DNF_size >= 2 ** (VC_dimension / 4),
    "counterexample": "" if DNF_size >= 2 ** (VC_dimension / 4) else f"DNF size {DNF_size} < 2 ** (VC_dimension / 4)"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={math.sqrt(sum((result['metric_value'] - mean_value) ** 2 for result in results) / len(results))}")
    else:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"DNF size too small\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

le': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1, 'counterexample': ''}

```

```

TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'VC-dimension', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
RESULT: SUPPORTED mean=1.0 std=0.0 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers instances with VC-dimension 1, which is a trivial sub-case, and the number of instances tested is very small ($n = 1$ for each trial). This does not provide sufficient evidence to support the conjecture for larger values of n or VC-dimension. The metric definition and construction gap are also potential concerns, as the test may not accurately measure the VC-dimension or the size of the DNF formula.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test only considers trivial sub-cases with VC-dimension 1 and a small number of instances, which does not provide sufficient evidence to support the conjecture. | next: Increase the number of instances tested and consider larger values of n and VC-dimension to provide more robust evidence for the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	cerebras	qwen-3-235b-a22b-ins	0	0	5914
2	preregistration	groq	llama-3.3-70b-versat	0	0	4413
3	novelty	groq	llama-3.3-70b-versat	0	0	4163
4	novelty	groq	llama-3.3-70b-versat	0	0	4525
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15000
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8040
7	critic	groq	llama-3.3-70b-versat	0	0	5111
8	judge	groq	llama-3.3-70b-versat	0	0	4246

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 51413 ms total latency. Provider mix: {'cerebras': 1, 'groq': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/91f293427310.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/91f293427310.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/91f293427310.tar.gz (if generated)